Introduction to Method Overriding and Recursion

INPUT/OUTPUT AND STREAMS IN JAVA

Alex Liu
Software Development Engineer

datacamp

Defining a base class

- The `Animal` class represents a generic animal

```
class Animal {
  void makeSound() {
      System.out.println("Animal sound");
  }
  public static void main(String[] args) {
      Animal a = new Animal();
      a.makeSound(); // Output: Animal sound
  }
}
```

```
Animal sound
```

Understanding inheritance-extends

```
// Use `extends` to create a subclass of `Animal` named `Dog`
class Dog extends Animal {
    void bark() {System.out.println("Bark");}
}
public static void main(String[] args) {
    Dog d = new Dog(); // Create new instance of Dog object which extends Animal
    d.makeSound();
    d.bark();
}
```

- Print both sound line by line:

```
Animal sound
Bark
```

Overriding methods in Java

```
class Cat extends Animal {
    @Override // Use `@Override` to override the behavior of `.makeSound()`
    void makeSound() {
        System.out.println("Meow");
    }
}
public static void main(String[] args) {
    Cat c = new Cat(); // Create a new instance of Cat object which extends Animal
    c.makeSound(); // Call the overrided method `makeSound()`
}
```

```
Meow
```

Understanding recursion

- A method calling itself to solve a problem
- Requires a **base case** to prevent infinite recursion

```
public class RecursionExample {
    static void countdown(int n) {
        // Base case
        if (n == 0) return;
        System.out.println(n);
        // Recursive call
        countdown(n - 1);
    }
}
```

Recursion sample usage

- Sample usage

```
public static void main(String[] args) {
  countdown(5);
}
```

- Recursively call stop when it reach the **base case** which is `0`

```
5
4
3
2
1
```

Summary

- `extends` enables inheritance
- `@Override` modifies inherited behavior
- `Recursion` breaks problems into smaller steps
  - **Base case** prevents infinite recursion

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Introduction to Date and Time in Java

INPUT/OUTPUT AND STREAMS IN JAVA

Alex Liu
Software Development Engineer

datacamp

Creating Date objects

- The `LocalDate` class
  - Represents dates without time
- The `LocalTime` class
  - Represents time without date
- Import required classes

```
import java.time.LocalDate;
import java.time.LocalTime;
```

Creating Date objects(continued)

- Retrieve current date and time using `.now()`

```
LocalDate date = LocalDate.now();
LocalTime time = LocalTime.now();
System.out.println(date);
System.out.println(time);
```

- (Note: The time will vary based on when the program is executed)

```
2025-03-10
12:45:30.123456
```

Formatting Dates in Java

- `DateTimeFormatter`
  - Define custom date formats
  - Supports multiple patterns like `yyyy-MM-dd`, `MM/dd/yyyy`
- Import necessary classes

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
```

Formatting Dates in Java(continued)

- Create a `LocalDate` instance `date`

```
LocalDate date = LocalDate.now(); // Current format we have will be: `2025-03-10`
```

- Create a `DateTimeFormatter` with the pattern `MM/dd/yyyy` using `.ofPattern()`

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
```

- Apply the `DateTimeFormatter` using `.format()` method

```
System.out.println(date.format(formatter))
```

```
03/10/2025
```

Parsing String into Date

- Using `LocalDate.parse()` to converts text to a date
  - Ensure the format matches the input string
- Use `.parse()` with a text in `yyyy-MM-dd` format to parse into a `Date` object

```
LocalDate parsedDate = LocalDate.parse("2024-03-10");
System.out.println(parsedDate);
```

- Print the `Date` object will output:

```
2024-03-10
```

Performing Date adjustment

- Use `.plusDays()` and `.minusDays()` to adjust dates

```
LocalDate date = LocalDate.now();
// Current value:
System.out.println(date);
// Apply `.plusDays()` with value `7`
LocalDate futureDate = date.plusDays(7);
// Value after adjustment
System.out.println(futureDate);
```

```
2025-03-10
2025-03-17
```

Performing Date adjustment(continued)

- Apply `.minusDays()` with value `7`

```
LocalDate pastDate = date.minusDays(7);
System.out.println(pastDate);
```

- Current value for `pastDate` :

```
2025-03-03
```

Summary

- `LocalDate` and `LocalTime` manage dates and times separately
  - Note: The time will vary based on when the program is executed
  - `.parse()` can converts text to a date
  - `.now()` retrieves the current date/time
- `DateTimeFormatter` formats and parses dates
  - define custom date formats
  - Supports multiple patterns like `yyyy-MM-dd` , `MM/dd/yyyy`
- Perform date calculations using `.plusDays()` and `.minusDays()`

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Introduction to Enums

INPUT/OUTPUT AND STREAMS IN JAVA

Alex Liu
Software Development Engineer

datacamp

Why Use Enums?

- Enums define a fixed set of values
  - Example: Days of the week, Status codes
- Improves code readability and maintainability
- Prevents invalid values in variables
- Notes: No import is needed, as `Enums` are part of core Java library

Defining an Enum

- Declare an `Enum` using `enum` keyword

```
enum Day {
  MONDAY,
  TUESDAY,
  WEDNESDAY,
  THURSDAY,
  FRIDAY,
  SATURDAY,
  SUNDAY;
}
```

- Each value is a constant and written in uppercase

Using Enums in a program

- Example: Assign and print an Enum value

```
public class EnumExample {
  public static void main(String[] args) {
      Day today = Day.WEDNESDAY; // Assigning an Enum value
      System.out.println("Today is: " + today);
  }
}
```

- Output:

```
Today is: WEDNESDAY
```

Looping through Enum values

- Use `.values()` to get all values in an Enum
- Use `.ordinal()` return the position of the element
- Lets reuse the previous defined `Day` `Enum` as an example:

```
for (Day d : Day.values()) {
  System.out.println(d);
  System.out.print(
  " is at index " + d.ordinal());
}
```

- This program will output all the values defined in the `Enum` :

```
MONDAY is at index 0
TUESDAY is at index 1
WEDNESDAY is at index 2
THURSDAY is at index 3
FRIDAY is at index 4
SATURDAY is at index 5
SUNDAY is at index 6
```

Enum with methods

- `Enums` can have methods for additional functionality

```
enum Status {
    SUCCESS("Operation successful"),
    ERROR("An error occurred");

    private String message;
    Status(String message) {
        this.message = message;
    }
    public String getMessage() {
        return message;
    }
}
```

Enum with methods

- Example usage of `Enum` named `Status`

```
public class EnumMethodExample {
    public static void main(String[] args) {
        Status current = Status.SUCCESS;
        System.out.println("Status: " + current);
        System.out.println("Message: " + current.getMessage());
    }
}
```

- Output:

```
Status: SUCCESS
Message: Operation successful
```

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Custom methods - Bringing it all together

INPUT/OUTPUT AND STREAMS IN JAVA

Alex Liu
Software Development Engineer

datacamp

Integrating Concepts

- Combine and use:
  - Enums
  - Date and Time
  - Recursion
- Goal: build clear, reusable, and maintainable methods
- Example program:
  - An e-commerce site
    - with practical `OrderStatus` system with order tracking methods
    - with loyalty discount algorithm.

Defining Enum with methods

- Define `OrderStatus` `Enum` to represent order states

```
enum OrderStatus {
    PLACED("Order placed successfully"),
    DELIVERED("Order delivered");
    private final String message;
    OrderStatus(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }
}
```

Custom method for Date operations

- Use `orderDate` to estimate when the order will deliver to customer
  - User .plusDays() method adds days to a given date

```
import java.time.LocalDate;

public class OrderUtils {
    public static LocalDate estimateDelivery(LocalDate orderDate,
                                             int daysToDeliver) {
        return orderDate.plusDays(daysToDeliver);
    }
}
```

Recursive method for discount

- **Reward policy:** Incrementing discounts month-by-month to reward return customers
  - Use recursive method to calculate the cumulative discount:

```
public class DiscountCalculator {
    public static double cumulativeDiscount(int months) {
        if (months <= 0) return 0;
        return 5 + cumulativeDiscount(months - 1); // $5 discount per month
    }
}
```

Custom object

- Create an Order class which contain all order related information
  - Contain important information about order like **id**, **date** and **current status**.

```
class Order {
    int orderId;
    OrderStatus orderStatus;
    LocalDate orderDate;

    Order(int orderId, LocalDate orderDate) {
        this.orderId = orderId;
        this.orderDate = orderDate;
        this.orderStatus = orderStatus;
    }
```

Customer object with toString() method

- We want to print the order in a way that the employee is easy to read or process
  - Define `toString()` method to define how the `Order` object will be printed.

```
public String toString() {
        return "Order[id=" + orderId + ", date=" + orderDate + "]";
    }
}
```

- Sample output by calling .toString():

```
Order[id=10015, date=2025-03-19]
```

Main method

```
public class Main {
    public static void main(String[] args) {
        Order order = new Order(101, LocalDate.now());
        System.out.println(order.toString());

        LocalDate delivery = OrderUtils.estimateDelivery(order.orderDate, 7);
        System.out.println("Estimated Delivery: " + delivery);
        System.out.println(OrderStatus.PLACED);

        double discount = DiscountCalculator.cumulativeDiscount(3);
        System.out.println("Total Discount: $" + discount);
    }
}
```

Main method - output

- This main method is executed when a new order is placed with following information:
  - The user has subscribed for 3 months already
  - The estimate delivery time is 7 days
  - Order status will initially be `PLACED`

```
Order[id=101, date=2025-03-09]
Estimated Delivery: 2025-03-16
Order placed successfully
Total Discount: $15.0
```

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Congratulations!

INPUT/OUTPUT AND STREAMS IN JAVA

Alex Liu
Software Development Engineer

datacamp

Chapter 1 Wrap-Up: Java File Operations

- Java file operations
  - Creating/deleting files
  - Checking file existence
- Reading and writing text files
  - `FileReader`, `FileWriter`
  - Efficient file processing
- Directory management
  - Creating directories (`mkdir()`)
  - Listing files (`listFiles()`)
  - Working with paths (`getPath()`)

Chapter 2 Wrap-up: Working with Iterators and Streams

- Traversing data collections
  - Iterators (`Iterator`, `ListIterator`)
  - Safe element removal/modification
- Data transformation with Streams
  - Converting elements (`map()`)
  - Filtering data (`filter()`)
  - Storing results (`collect()`)
- Aggregating data
  - Combining results (`reduce()`)
  - Finding sums and totals
  - Improving readability and efficiency

Chapter 3 wrap-up: Custom Methods, Dates, and Enums

- Method overriding and recursion
  - Specific behaviors (`@Override`)
  - Solving repetitive problems (`recursion`)
- Managing dates and time
  - Current dates/times (`LocalDate`, `LocalTime`)
  - Formatting dates (`DateTimeFormatter`)
  - Performing calculation(`plusDays()`)
- Working with Enums
  - Defining constants
  - Improving code clarity
  - Custom Enum methods and fields
- Creating reusable custom methods
  - Integrating concepts effectively
  - Building maintainable applications

Continue your journey on Java

- Internet Sources
  - **Baeldung**: website dedicated to Java topics and assistance
  - **Oracle's Java site**: the stewards of Java technology (Oracle) web site
  - **dev.java**: Java developer news, events, and other information
  - **Open JDK**: community dedicated to a free and open-source implementation of Java
- Books
  - *Head First Java* by Sierra et al
  - *Learn Java In One Day* by Jamie Chan
  - *Java for Beginners* by Brady Ellison
- Further resources:
  - **Java project ideas**

Let's keep learning

INPUT/OUTPUT AND STREAMS IN JAVA